Artificial Intelligent - Report Lab 1

Anh Khoi Pham - ITCSIU23018

March 9, 2026

1 Idea

To ensure the vacuum agent can discover and clean all dirt cells, it must continuously explore and update its internal map of the environment. The agent builds a grid-based map by dynamically marking cells as `CLEAR`, `WALL`, or `DIRT` based on its percepts at each step.

For each iteration, the agent operates using the following logic:

1. **Clean:** If the current cell contains dirt, the agent immediately issues a `SUCK` action.
2. **Explore:** If the current cell is clean, the agent identifies the nearest frontier cell (an `UNKNOWN` cell adjacent to a known cell) using Breadth-First Search (BFS). BFS guarantees finding the closest unvisited area in terms of grid distance.
3. **Navigate:** Once a frontier is identified, the agent uses the A* search algorithm with a Manhattan distance heuristic to find the optimal path to it, actively avoiding known walls.
4. **Execute Path:** The `follow_path` function translates the sequence of grid coordinates returned by A* into physical movements (`FORWARD`, `TURN_LEFT`, or `TURN_RIGHT`) based on the agent's current directional heading.
5. **Return Home:** If no frontiers are left, meaning the reachable environment is fully explored and cleaned, the agent uses A* to pathfind back to its starting cell (Home).

Here is the pseudo-code for the overall process:

```
if see dirt:
    do clean

frontier <- find_frontier() # BFS to find the next nearest unknown cell

if can find frontier:
    # Expand the internal map by navigating to the frontier
    path <- a_star(current_position, frontier)
    do follow_path(path)
else if not at home:
```

```
    # Since we explored all possible frontiers -> environment is clean -> return home
    path <- a_star(current_position, home)
    do follow_path(path)
else:
    # Already at home and done
    do nothing
```

You can see the Python implementation of this pseudo-code in the `execute()` function of `myvacuumagent.py`, supported by the utility functions `a_star()` and `find_frontier()`.

2 Performance Evaluation

My Vacuum Performance

With the `iteration_counter = height * width * 2`, My Vacuum agent gives a high performance score of 650.0, successfully returning to home after cleaning all the dirt. This excellent result is attributed to the agent's memory-based mapping and pathfinding, which prevents it from endlessly wandering into already cleaned areas. However, it is worth noting that this iteration limit configuration does not absolutely guarantee a successful return home in highly complex or maze-like map layouts, as excessive backtracking might require more steps than allocated.

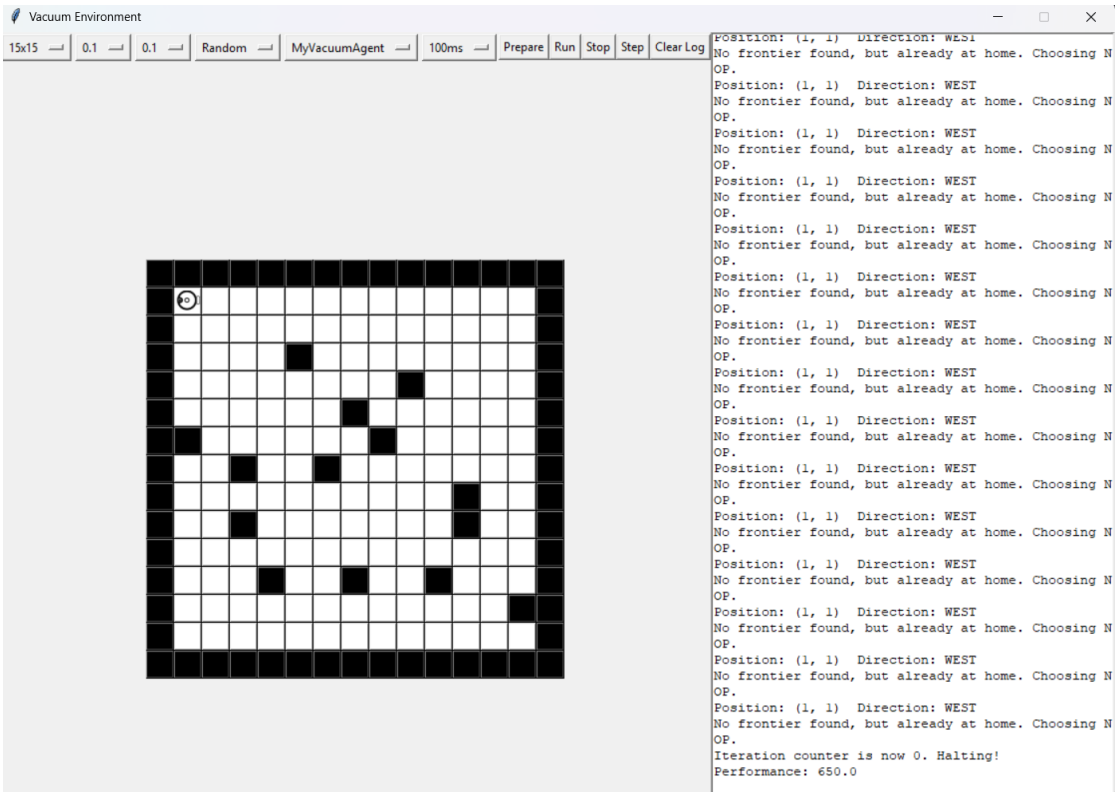

Figure 1: My Vacuum Performance

Reactive Agent Performance

Compared to My Vacuum agent, the default Reactive agent produces a significantly more modest result, achieving a performance score of only 55.0 and failing to clean all the dirt in the environment. This massive difference highlights the limitation of a memoryless approach: without an internal map, the Reactive agent

relies heavily on randomized or purely localized logic, causing it to get stuck in loops, miss entire sections of the map, and waste iterations.

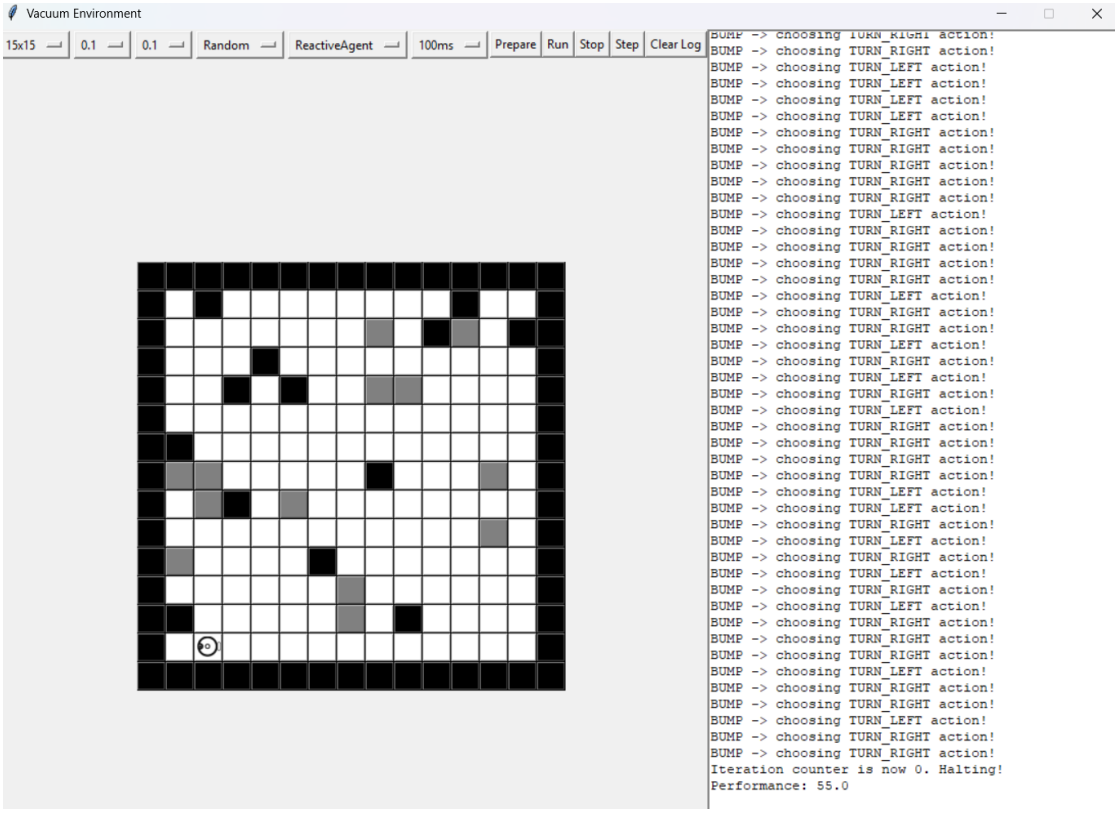

Figure 2: Reactive Agent Performance